21720 Programació concurrent

5. Semàfors

Introducció

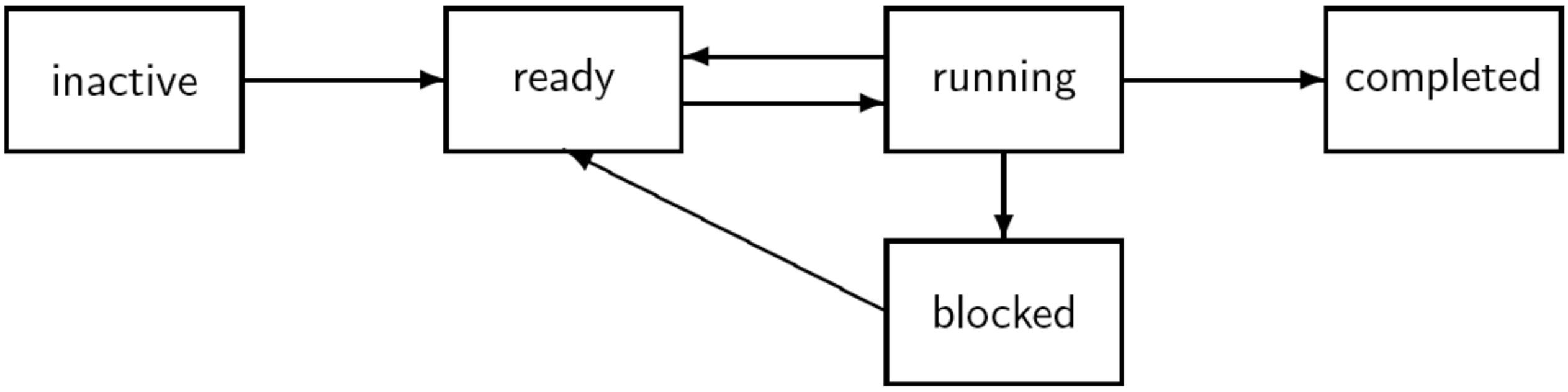
- El concepte de semàfor introduït per Edsger W. Dijkstra 1974 inspirant-se en els senyals visuals ferroviaris

- Usat per solucionar problemes de sincronització sense espera activa

- Les solucions anteriors funcionen sobre la màquina sense col·laboració del SO, els semàfors s'implementen com a serveis del nucli del SO

Estats dels processos

- **Preparat** (*ready*): Un procés que “vol executar-se”, pot executar la següent instrucció
- **Executant-se** (*running*)
- **Bloquejat** (*blocked*): No està preparat i no és candidat a executar-se
- **Desbloquejat** (*unblocked*) o despertat (*awakened*) o alliberat (*released*):



Definició

- Un **semàfor** *S* es defineix per dues components: *S.V* un enter no negatiu (valor del semàfor) i *S.L* un conjunt de processos (els bloquejats pel semàfor)

- La inicialització es pot fer amb un valor $k \geq 0$ per *S.V* i un conjunt buit per *S.L*

semàfor $S \leftarrow (k, \emptyset)$

Operacions atòmiques

***wait*(S)**

```
if S.V > 0
   S.V ← S.V - 1
else
   S.L ← S.L ∪ p
   p.estat ← bloquejat
```

***signal*(S)**

```
if S.L = 0
   S.V ← S.V + 1
else
   S.L ← S.L - {q}
   q.estat ← preparat
```

- La notació original és P(S) “Prolaag” per *wait* i V(S) “Verhoog” per *signal*. *p* denota el procés que executa la instrucció i *q* és un element arbitrari del conjunt de processos bloquejats a S.L

- ***wait*** o ***acquire*** o ***lock***: Si el valor del semàfor no és zero decrementa el valor i *p* continua l’execució en cas contrari *p* s’afegeix a la cua i queda bloquejat a S

- ***signal*** o ***release*** o ***post***: Si la cua és buida s’incrementa el valor en cas contrari es desbloqueja un procés arbitrari de la cua. L’estat de *p* no canvia

Semàfor general o comptador ***multiplex***

- Semàfor com un comptador dels recursos disponibles o permisos (*permits* a Java)

- Comptador: nombre de cridades a *wait* que es poden fer sense bloquejar el procés

- Exclusió mútua: Un procés a la SC per tant valor del semàfor 1

Semàfor binari ***mutex***

- És aquell tal que el seu valor enter només pot ser 1 o 0
- El terme *mutex* s'usa als *pthreads* i a *java.util.concurrent*
- S'inicialitza amb (0,Ø) o (1,Ø)
- *wait*(S) no canvia
- *signal*(S) es defineix com:

***signal*(S)**

```
if S.V = 1
   //no definit
else if S.L = 0
   S.V ← 1
else // com abans
   S.L ← S.L - {q}
   q.estat ← preparat
```

Algorisme de Bartz

- Simulació de semàfors generals amb binaris
- Amb dos semàfors binaris *S* i *gate* i una variable entera *count*
- *S* assegura l'exclusió mútua per a l'accés a *count*
- *gate* s'usa per als processos que s'ha de bloquejar o desbloquejar segons *count*

Algorithm 6.13: Barz's algorithm for simulating general semaphores

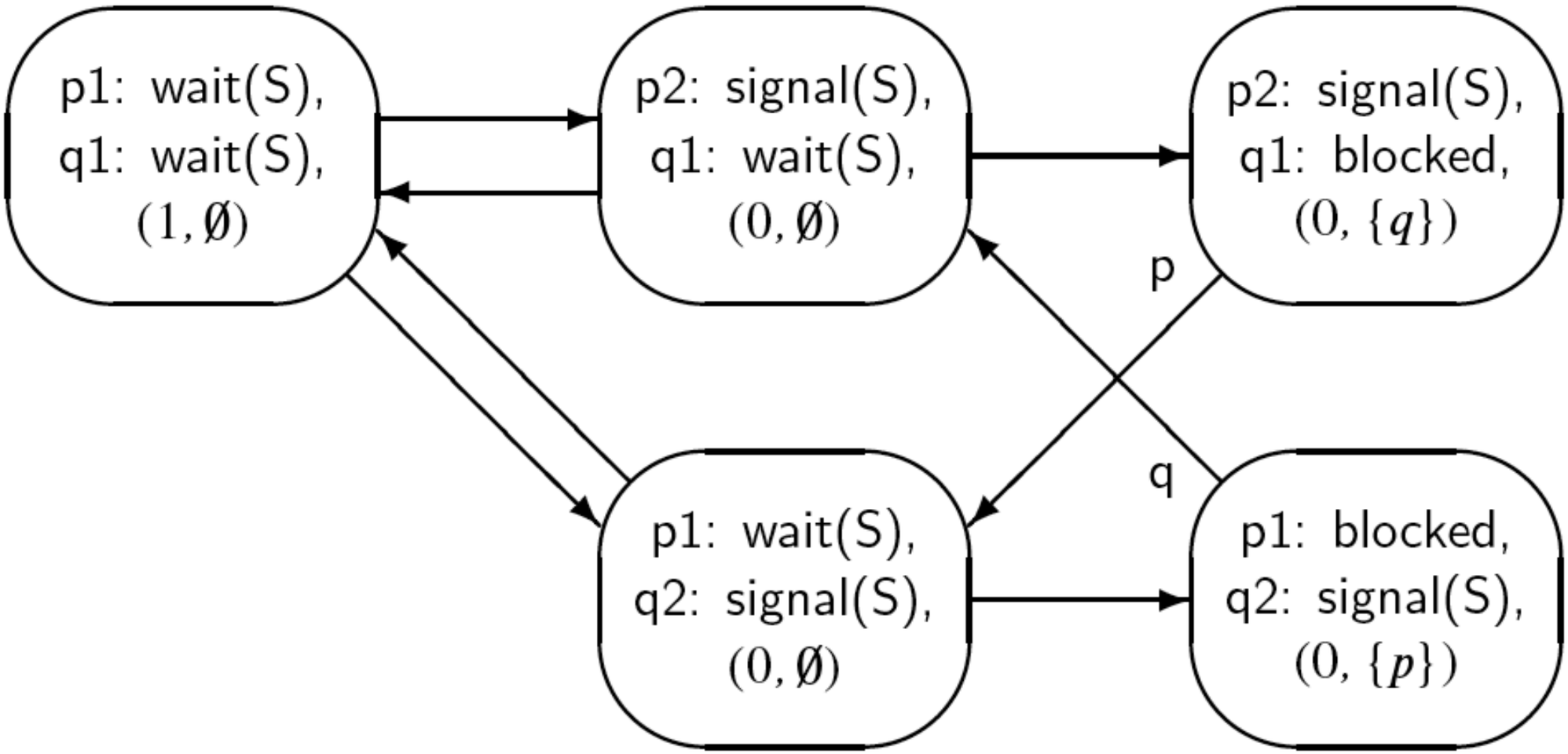
```
                                binary semaphore S ← 1
                                binary semaphore gate ← 1
                                integer count ← k
loop forever
    non-critical section
p1:     wait(gate)              // Si no és el primer en entrar a la SC ha d'esperar
p2:     wait(S)                                   // Simulated wait
p3:     count ← count − 1
p4:     if count > 0 then
p5:         signal(gate)        // Permet que entri l'altre si el valor és positiu
p6:     signal(S)
    critical section
p7:     wait(S)                                   // Simulated signal
p8:     count ← count + 1
p9:     if count = 1 then       // count s'incrementa i permet l'entrada d'un altre procés
p10:        signal(gate)
p11:    signal(S)
```

Secció crítica amb semàfors

Algorithm 6.1: Critical section with semaphores (two processes)	
binary semaphore S $\leftarrow (1, \emptyset)$	
p	**q**
loop forever	loop forever
p1: non-critical section	q1: non-critical section
p2: wait(S)	q2: wait(S)
p3: critical section	q3: critical section
p4: signal(S)	q4: signal(S)

- *p* desitja entrar a la SC el preprotocol és *wait*(S) si S.V = 1 es decrementa i *p* entra a la SC quan *p* en surt el postprotocol és *signal*(S) i el valor de S.V torna a ser 1

- Si *q* intenta entrar a la SC executant *wait*(S) abans que *p* hagi sortit S.V = 0 i *q* queda bloquejat a S.L per tant (0,{*q*}). Quan *p* surti *q* es desbloquejat i entra a la SC



- Garanteix l'**exclusió mútua**: ja que no es pot arribar a cap estat com (*p2*: *signal*(S), *q2*: *signal*(S),…)

- No hi ha **interbloqueig**: no hi ha cap estat amb els dos processos bloquejats

- No hi **espera infinita**: quan un procés executa un *wait* (deixa la secció no crítica) passa a estar bloquejat i la única manera de continuar és amb un *signal*

semaphore.c

Implementació amb *C*

- Solució amb **POSIX semaphores**: Inicialització obsoleta per OS X

https://docs.oracle.com/cd/E19455-01/806-4750/svipc-75667/index.html

- Solució amb **semàfors GCD**

https://developer.apple.com/library/ios/documentation/Performance/Reference/GCD_libdispatch_Ref/

```
int counter = 0;
//sem_t  mutex; // Used for a unnamed POSIX semaphore
// Semàfors de GCD per a iOS i OSX
dispatch_semaphore_t mutex;

void *count(void *ptr) {
    long i, max = MAX_COUNT/NUM_THREADS;
    int tid = ((struct tdata *) ptr)->tid;

    for (i=0; i < max; i++) {
        //sem_wait(&mutex);
        dispatch_semaphore_wait(mutex, DISPATCH_TIME_FOREVER);
        counter += 1;
        //sem_post(&mutex);
        dispatch_semaphore_signal(mutex);
    }
    printf("End %d counter: %d\n", tid, counter);
}
```

```
int main (int argc, char *argv[]) {
    pthread_t threads[NUM_THREADS];
    int rc, i;
    struct tdata id[NUM_THREADS];
    // Semaphore initialization DEPRECATED per a SO X
    //sem_init(&mutex, 0, 1);
    mutex = dispatch_semaphore_create(1); // init with value of 1
```

mutex.c

```c
int counter = 0;
/* POSIX Thread mutex */
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
/* Don't confuse it with POSIX Semaphore */

void *count(void *ptr) {
    long i, max = MAX_COUNT/NUM_THREADS;
    int tid = ((struct tdata *) ptr)->tid;

    for (i=0; i < max; i++) {
        pthread_mutex_lock(&mutex);
        counter += 1;
        pthread_mutex_unlock(&mutex);
    }
    printf("End %d counter: %d\n", tid, counter);
}
```

Implementació amb *C* (cont.)

- Solució amb **POSIX Thread mutex**
- https://linux.die.net/man/3/pthread_mutex_lock
- http://stackoverflow.com/questions/2065747/pthreads-mutex-vs-semaphore

CounterSemaphore.java

```java
static volatile int counter = 0;
static Semaphore mutex = new Semaphore(1);
int id;

public CounterSemaphore(int id) {
    this.id = id;
}

@Override
public void run() {
    int max = MAX_COUNT / THREADS;
    System.out.printf("Thread %d\n", id);
    for (int i = 0; i < max; i++) {
        try {
            mutex.acquire();
        } catch (InterruptedException e) {
        }
        counter += 1;
        mutex.release();
    }
}
```

- Implementació amb Java

  http://docs.oracle.com/javase/7/docs/api/java/util/concurrent/Semaphore.html

countersemaphore.py

```
mutex = threading.Semaphore(1)
counter = 0

def thread():
    global counter

    print("Thread {}".format(threading.current_thread().name))

    for i in range(MAX_COUNT/THREADS):
        mutex.acquire()
        counter += 1
        mutex.release()
```

- Implementació amb Python

  https://docs.python.org/3/library/threading.html

- Implementació amb Go: https://godoc.org/golang.org/x/sync/semaphore

Problemes d'ordre d'execució

- Secció crítica: Abstracció del problema de sincronització de diferents processos competint pel mateix recurs

- Altres problemes de sincronització: Ordre d'execució d'operacions de diferents processos

Algorithm 6.5: Mergesort

integer array A
binary semaphore S1 $\leftarrow (0, \emptyset)$ // Inicialitzat a 0 merge està bloquejat a S1
binary semaphore S2 $\leftarrow (0, \emptyset)$

sort1	**sort2**	**merge**
p1: sort 1st half of A	q1: sort 2nd half of A	r1: wait(S1)
p2: signal(S1)	q2: signal(S2)	r2: wait(S2)
p3:	q3:	r3: merge halves of A

Barreres

- Mecanismes de sincronització que obliga a que els processos concurrents hagin d'arribar a un determinat punt per continuar

- **Barrera**: Conjunt de punts de sincronització

- Els semàfors de l'algorisme de Mergesort són barreres. El valor de cada semàfor indica si el seu procés ha arribat a la meta

Problema de productors-consumidors

- Exemple del problema d'ordre d'execució
- Problema present a la majoria de mecanismes de comunicació: *pipeline* entre processos i comandes, E/S de dispositius, comunicacions per xarxa…
- Processos **Productors**: Produeixen elements que són enviats als consumidors
- Processos **Consumidors**: Reben els elements i els consumeixen

- **Síncrons**: Quan es produeix un element ha de ser consumit per que el productor continuï

- **Asíncrons**: El canal de comunicació té capacitat d'emmagatzematge *buffer*. Els productors afegeixen a la cua i el consumidors agafen

- Problemes de sincronització amb *buffers*: Un consumidor no pot agafar d'un *buffer* buit un productor no pot afegir dades a un *buffer* ple

- ***Buffer* infinit**: Tenint en compte les velocitats relatives dels productors un *buffer* de mida raonable no s'omplirà mai

Algorithm 6.6: Producer-consumer (infinite buffer)

infinite queue of dataType buffer ← empty queue
semaphore notEmpty ← $(0, \emptyset)$

producer	**consumer**
dataType d	dataType d
loop forever	loop forever
p1: d ← produce	q1: wait(notEmpty)
p2: append(d, buffer)	q2: d ← take(buffer)
p3: signal(notEmpty)	q3: consume(d)

- Amb la simplificació del *buffer* infinit només s'ha de controlar que el *buffer* no estigui buit

- L'algorisme suposa que les parelles d'instruccions *p2* i *p3* i *q1* i *q2* són atòmiques

- Si no es fa la suposició s'ha d'afegir un altre semàfor que garanteixi l'exclusió mútua quan s'insereixin o es llevin elements

producerconsumerinfinite.py

```python
TO_PRODUCE = 1000
PRODUCERS = 2
CONSUMERS = 2

buffer = []
mutex = threading.Semaphore(1)
notEmpty = threading.Semaphore(0)

def producer():
    id = threading.current_thread().name
    print("Producer {}".format(id))
    for i in range(TO_PRODUCE):
        data = "{} i: {}".format(id, i)
        mutex.acquire()
        buffer.append(data)
        print("        {} PRODUEIX: {}".format(id, data))
        mutex.release()
        notEmpty.release()

def consumer():
    id = threading.current_thread().name
    print("Consumer {}".format(id))
    for i in range(TO_PRODUCE):
        notEmpty.acquire()
        mutex.acquire()
        data = buffer.pop(0)
        print("{} CONSUMEIX: {}".format(id, data))
        mutex.release()
```

```python
TO_PRODUCE = 1000
PRODUCERS = 2
CONSUMERS = 2

buffer = []
mutex = threading.Lock()
notEmpty = threading.Semaphore(0)

def producer():
    id = threading.current_thread().name
    print("Producer {}".format(id))
    for i in range(TO_PRODUCE):
        data = "{} i: {}".format(id, i)
        with mutex:
            buffer.append(data)
        print("        {} PRODUEIX: {}".format(id, data))
        notEmpty.release()

def consumer():
    id = threading.current_thread().name
    print("Consumer {}".format(id))
    for i in range(TO_PRODUCE):
        notEmpty.acquire()
        with mutex:
            data = buffer.pop(0)
            print("{} CONSUMEIX: {}".format(id, data))
```

- Implementació amb python usant dos Semaphore i amb un Lock i un Semaphore

http://www.laurentluce.com/posts/python-threads-synchronization-locks-rlocks-semaphores-conditions-events-and-queues/

Algorithm 6.8: Producer-consumer (finite buffer, semaphores)

finite queue of dataType buffer ← empty queue
semaphore notEmpty ← $(0, \emptyset)$
semaphore notFull ← $(N, \emptyset)$

producer	**consumer**
dataType d	dataType d
loop forever	loop forever
p1: d ← produce	q1: wait(notEmpty)
p2: wait(notFull)	q2: d ← take(buffer)
p3: append(d, buffer)	q3: signal(notFull)
p4: signal(notEmpty)	q4: consume(d)

- Si el *buffer* és finit s'haurà d'afegir un altre semàfor per controlar que no estigui ple
- Es pot emprar un semàfor comptador *notFull* que indiqui les posicions lliures que originalment s'inicialitza amb la mida del *buffer*
- **Semàfors partits**: Dos o més semàfors que satisfan un invariant tal que la suma dels seus valors és un nombre fixat *N*

notEmpty + notFull = *N* (mida del *buffer*)

- Els semàfors partits no són un tipus de semàfors és un terme per descriure el mecanisme de sincronització

- **Semàfors forts**: la cua de processos bloquejats és FIFO i asseguren l'espera limitada i equitat

  A Java: `Semaphore sem = new Semaphore(1, true);`

- **Semàfors dèbils**: no es garanteix cap ordre concret d'alliberament dels processos bloquejats.

producerconsumer.py

```
TO_PRODUCE = 1000
BUFFER_SIZE = 10
PRODUCERS = 2
CONSUMERS = 2

class ProducerConsumer(object):
    def __init__(self, size): #constructor
        #deque doble cua permet treure i afegir left right
        self.buffer = collections.deque([], size)
        self.mutex = threading.Lock()
        self.notFull = threading.Semaphore(size)
        self.notEmpty = threading.Semaphore(0)

    def append(self, data):
        self.notFull.acquire()
        with self.mutex:
            self.buffer.append(data)
        self.notEmpty.release()

    def take(self):
        self.notEmpty.acquire()
        with self.mutex:
            data = self.buffer.popleft()
        self.notFull.release()
        return data
```

```
def producer(buffer):
    id = threading.current_thread().name
    print("Producer {}".format(id))
    for i in range(TO_PRODUCE):
        data = "{} i: {}".format(id, i)
        buffer.append(data)
        print("        {} PRODUEIX: {}".format(id, data))

def consumer(buffer):
    id = threading.current_thread().name
    print("Consumer {}".format(id))
    for i in range(TO_PRODUCE):
        data = buffer.take()
        print("{} CONSUMEIX: {}".format(id, data))
```

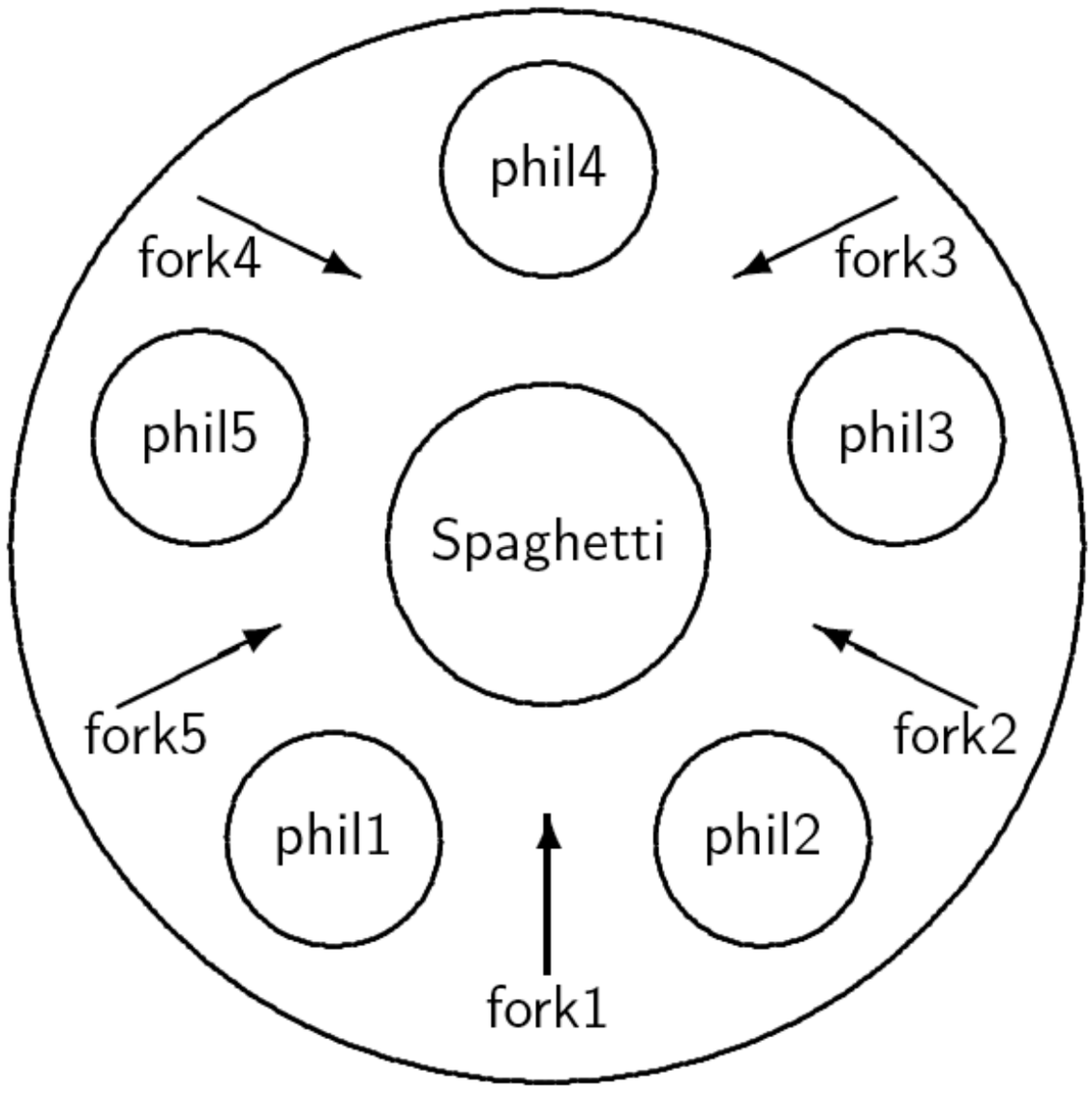
El problema del sopar dels filòsofs

- Serveix per a l'estudi i comparació dels diferents mecanismes de sincronització

- Hi ha 5 filòsofs que només poden menjar o pensar

- A la taula el menjar es comparteix, es tenen 5 plat i cinc bastonets i al centre un bol d'espaguetis que no s'acaba mai

- Per menjar es necessiten dos bastonets i només es poden agafar els que es tenen al costat i només un a la vegada



Algorithm 6.9: Dining philosophers (outline)

```
      loop forever
p1:       think
p2:       preprotocol
p3:       eat
p4:       postprotocol
```

- El pre i postprotocol asseguren que el filòsofs només menja si es compleixen els requeriments:

- El filòsof només menja si té dos bastonets
- Exclusió mútua: dos filòsofs no poden emprar el mateix bastonet a la vegada
- S'ha d'assegurar el progrés (no hi interbloqueig)
- S'ha d'assegurar l'espera limitada (no hi ha inanició)
- Ha de ser eficient: En absència de competència el bastonet ha de poder-se usar per un dels veïns

Algorithm 6.10: Dining philosophers (first attempt)

semaphore array [0..4] fork ← [1,1,1,1,1]

```
      loop forever
p1:       think
p2:       wait(fork[i])
p3:       wait(fork[i+1])
p4:       eat
p5:       signal(fork[i])
p6:       signal(fork[i+1])
```

- Filòsofs i bastonets estan inicialitzats amb els índex de *0..4*

- El $phil_0$ té a l'esquerra el $fork_0$ i a la dreta el $fork_1$ i així successivament fins al $phil_4$ que té a l'esquerra el $fork_4$ i a la dreta el $fork_0$

- Cada bastonet està modelat com un semàfor: *wait* es correspon a agafar un bastonet i el *signal* a amollar-lo

- Abans de menjar cada filòsof fa *wait* sobre els bastonets que li corresponen (esquerra i dreta). Si algun d'ells està agafat quedarà bloquejat fins que el que el té faci el *signal* corresponent

- **Interbloqueig *deadlock***: Si tots els filòsofs intenten menjar simultàniament agafaran el bastonet de l'esquerra i quedaran bloquejats esperant el de la dreta

- Condicions necessàries per interbloqueig:

    1. **Exclusió mútua**: els recursos només es poden assignar a un procés

    2. **Retenció i espera (*hold and wait*)**: El procés manté el recurs assignat mentre n'espera un altre

    3. **No apropiació (*no preemption*)**: No es pot llevar un recurs assignat a un procés, ha de ser el procés el que l'alliberi

    4. **Espera circular (*circular wait*)**: Es produeix un bucle tancat de processos esperant recursos assignats a altres (derivat de 2)

- Per evitar l'interbloqueig basta evitar algun dels 4 punts anteriors

- Aplicació al problema dels filòsofs: Tots els processos han de sol·licitar els recursos en el mateix ordre (ascendent o descendent)

- El $phil_4$ agafa primer el $fork_0$ i després el $fork_4$! Per tant per ell es pot fer: Algorisme LR

Algorithm 6.12: Dining philosophers (third attempt)	
semaphore array [0..4] fork ← [1,1,1,1,1]	
	philosopher 4
	loop forever
p1:	think
p2:	wait(fork[0])
p3:	wait(fork[4])
p4:	eat
p5:	signal(fork[0])
p6:	signal(fork[4])

philosopherslr.py

```
class Philosopher(threading.Thread):
    forks = []
    count = 0

    def __init__(self):
        super(Philosopher, self).__init__()
        self.id = Philosopher.count
        Philosopher.count += 1
        Philosopher.forks.append(threading.Lock())

    def right(self):
        return (self.id + 1) % PHILOSOPHERS

    def pick(self):
        if self.id < self.right():
            # Pels filosofs 0..3
            Philosopher.forks[self.id].acquire()
            Philosopher.forks[self.right()].acquire()
        else:
            # Pel filosof 4
            Philosopher.forks[self.right()].acquire()
            Philosopher.forks[self.id].acquire()

    def release(self):
        Philosopher.forks[self.id].release()
        Philosopher.forks[self.right()].release()

    def think(self):
        time.sleep(0.05)

    def eat(self):
        print("{} start eat".format(self.id))
        time.sleep(0.1)
        print("{} end eat".format(self.id))

    def run(self):
        for i in range(EAT_COUNT):
            self.think()
            self.pick()
            self.eat()
            self.release()
```

- Amb l'algorisme LR es poden donar una situació no òptima en la que només menja un filòsof

```
fork[0].wait()
  fork[1].wait()
    fork[2].wait()
      fork[3].wait()
        fork[0].wait()

fork[1].wait()
  fork[2].wait()
    fork[3].wait()
      fork[4].wait()
```

El $phil_4$ fa el *wait* en ordre decreixent i es bloqueja

El $phil_3$ agafa el $fork_4$ i menja (és l'únic)

Solució de Tannenbaum

- Plantejament com a sincronització d'instruccions: abans de menjar el filòsof ha de comprovar que cap dels seus veïns menja, si ho fan espera a que li notifiquin quan deixen els bastonets
- S'usa:
  - Un *array status* per designar l'estat del filòsof: pensant, amb fam, menjant
  - Un *array sync* per fer la sincronia entre els filòsofs
  - Un semàfor *mutex* per garantir l'exclusió mútua quan es verifica i manipula *status*

Dining philosophers (Tannenbaum solution)

```
semaphore mutex ← 1
semaphore array [0..4] sync ← [0,0,0,0]
semaphore array [0..4] status ← [THINK,THINK,THINKTHINK]
```

philosopher i

```
        loop forever
p1:         think
p2:         take_forks(i)
p3:         eat
p4:         drop_forks(i)
```

take_forks(i)

```
p2.1        wait(mutex)
p2.2        status[i] ← HUNGRY
p2.3        can_eat(i)
p2.4        signal(mutex)
p2.5        wait(sync[i])
```

drop_forks(i)

```
p4.1        wait(mutex)
p4.2        status[i] ← THINK
p4.3        can_eat(i-1)
p4.4        can_eat(i+1)
p4.5        signal(mutex)
```

can_eat(i)

```
p2.3.1      if (status[i] is HUNGRY and status[i-1] and status[i+1] is not EAT
                status[i] ← EAT
                signal(sync[i])
```

- La funció *take_forks* assigna l'estat a HUNGRY i amb la funció *can_eat* comprova si els veïns estan menjant si no és així ho senyalitza al semàfor *sync* i no es bloqueja

- Si cap dels dos veïns menja li assigna EAT a *status*

- Quan el filòsof acaba de menjar ha de verificar si els seus veïns estan esperant els bastonets que retenia

- Les cridades a *can_eat* sempre estan dins la secció crítica de *mutex*

philosophers.py

```
THINKING = 0
HUNGRY = 1
EATING = 2

class Philosopher(threading.Thread):
    mutex = threading.Lock()
    status = []
    sync = []
    count = 0

    def __init__(self):
        super(Philosopher, self).__init__()
        self.id = Philosopher.count
        Philosopher.count += 1
        Philosopher.status.append(THINKING)
        Philosopher.sync.append(threading.Semaphore(0))

    def right(self, i):
        return (i - 1) % PHILOSOPHERS

    def left(self, i):
        return (i + 1) % PHILOSOPHERS

    def canEat(self, i):
        if Philosopher.status[i] == HUNGRY and Philosopher.
            Philosopher.status[i] = EATING
            Philosopher.sync[i].release()
```

```
    def pick(self):
        Philosopher.mutex.acquire()
        Philosopher.status[self.id] = HUNGRY
        self.canEat(self.id)
        Philosopher.mutex.release()
        Philosopher.sync[self.id].acquire()

    def release(self):
        Philosopher.mutex.acquire()
        Philosopher.status[self.id] = THINKING
        self.canEat(self.left(self.id))
        self.canEat(self.right(self.id))
        Philosopher.mutex.release()

    def think(self):
        time.sleep(0.05)

    def eat(self):
        print("{} start eat".format(self.id))
        time.sleep(0.1)
        print("{} end eat".format(self.id))

    def run(self):
        for i in range(EAT_COUNT):
            self.think()
            self.pick()
            self.eat()
            self.release()
```

```
.status[self.left(i)] != EATING and Philosopher.status[self.right(i)] != EATING:
```

L'ós, el pot de mel i les abelles (Andrews 2000)

- Hi ha un ós que bàsicament menja mel d'un pot i dorm. Hi ha N abelles que carregen 1 porció de mel i la duen al pot. Al pot hi caben H porcions de mel. Les abelles duen mel al pot fins que està ple. Quan el pot està ple, la darrera abella desperta l'ós. L'ós es menja el pot sencer i mentre ho fa les abelles no el molesten. Quan acaba de menjar se'n va a dormir i les abelles comencen de bell nou.